

Experiment 2:

Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins.

Step 1: Install the Java JDK and maven, verify the installation

```
java -version
```

```
mvn -v
```

Step 2: Creating a Maven Project

- To create a basic Maven project using the command line, use the following command:

```
mvn archetype: generate \
```

```
-DgroupId=com.example \
```

```
-DartifactId=myapp \
```

```
-DarchetypeArtifactId=maven-archetype-quickstart
```

```
-DinteractiveMode=false
```

- Navigate to project directory

```
cd myapp
```

Step3: Understanding the POM File

The **POM (Project Object Model)** file is the XML file that contains all the configuration details about the project like project details, dependencies, build process and plugins.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>my-project</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <dependencies>
    <!-- Dependencies go here -->
  </dependencies>

  <build>
    <plugins>
      <!-- Plugins go here -->
    </plugins>
  </build>
</project>
```

- **<groupId>**: The group or organization that the project belongs to.
- **<artifactId>**: The name of the project or artifact.
- **<version>**: The version of the project (often follows a format like 1.0-SNAPSHOT).
- **<packaging>**: Type of artifact, e.g., jar, war, pom, etc.
- **<dependencies>**: A list of dependencies the project requires.
- **<build>**: Specifies the build settings, such as plugins to use.

Step 4: Dependency Management

- Maven uses the `<dependencies>` tag in the `pom.xml` to manage external libraries or dependencies that your project needs. When Maven builds the project, it will automatically download these dependencies from a repository (like Maven Central)

<dependencies>

<dependency>

<groupId>org.apache.commons</groupId>

<artifactId>commons-lang3</artifactId>

<version>3.12.0</version>

</dependency>

</dependencies>

Step 5: Using Plugins

Maven plugins are used to perform tasks during the build lifecycle, such as compiling code, running tests, packaging, and deploying. You can specify plugins within the `<build>` section of your **pom.xml**.(add this in your POM file)

```
<build>

<plugins>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.10.1</version>

<configuration>

    <source>17</source>

    <target>17</target>

</configuration>

</plugin>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-jar-plugin</artifactId>

<version>3.3.0</version>

<configuration>

<archive>

<manifest>

<mainClass>com.example.App</mainClass>

</manifest>

</archive>
```

</configuration>

</plugin>

</plugins>

</build>

Common Plugins

- **maven-compiler-plugin**: Compiles Java code.
- **maven-surefire-plugin**: Runs unit tests.
- **maven-jar-plugin**: Packages the project as a JAR file.
- **maven-clean-plugin**: Cleans up the `target/` directory.

Step 6: Run on Terminal

mvn clean install

Step 7: Command to Run Java Application

java -jar target/myapp-1.0-SNAPSHOT.jar